Pairing-Based SNARKs: Pinocchio and Groth16

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Recap

2 Encrypted Verification

3 Make It Sound

4 Make it Zero-Knowledge

5 Real Protocols

Recap

Recap: R1CS

Each **constraint** in the Rank-1 Constraint System must be in the form:

$$\langle \boldsymbol{\ell}, \mathbf{z} \rangle \times \langle \boldsymbol{r}, \mathbf{z} \rangle = \langle \boldsymbol{o}, \mathbf{z} \rangle$$

Where $\langle \mathbf{u}, \mathbf{v} \rangle$ is a dot product.

$$\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$$

Thus

$$\left(\sum_{i=1}^{n} \ell_i z_i \right) \times \left(\sum_{j=1}^{n} r_j z_j \right) = \sum_{k=1}^{n} o_k z_k$$

That is, actually, a quadratic equation with multiple variables.

Recap: R1CS

Consider the simplest program:

```
def example(a: F, b: F, c: F) -> F:
    if a:
        return b * c
    else:
        return b + c
```

Recap: R1CS

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

Thus, the next constraints can be built:

$$x_1 \times x_1 = x_1 \quad \text{(binary check)} \tag{1}$$
$$x_2 \times x_3 = \text{mult} \tag{2}$$
$$x_1 \times \text{mult} = \text{selectMult} \tag{3}$$
$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \tag{4}$$

The witness vector: $\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$.

The coefficients vectors:

$$\boldsymbol{\ell}_1 = (0,0,1,0,0,0,0), \quad \boldsymbol{r}_1 = (0,0,1,0,0,0,0), \quad \boldsymbol{o}_1 = (0,0,1,0,0,0,0)$$
$$\boldsymbol{\ell}_2 = (0,0,0,1,0,0,0), \quad \boldsymbol{r}_2 = (0,0,0,0,1,0,0), \quad \boldsymbol{o}_2 = (0,0,0,0,0,1,0)$$
$$\boldsymbol{\ell}_3 = (0,0,1,0,0,0,0), \quad \boldsymbol{r}_3 = (0,0,0,0,0,1,0), \quad \boldsymbol{o}_3 = (0,0,0,0,0,0,1)$$
$$\boldsymbol{\ell}_4 = (1,0,-1,0,0,0,0), \quad \boldsymbol{r}_4 = (0,0,0,1,1,0,0), \quad \boldsymbol{o}_4 = (0,1,0,0,0,0,-1)$$

Recap: QAP

R1CS provides us with the following constraint vectors:

$$\boldsymbol{\ell}_1, \boldsymbol{\ell}_2, \ldots, \boldsymbol{\ell}_m, \quad \boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_m, \quad \boldsymbol{o}_1, \boldsymbol{o}_2, \ldots, \boldsymbol{o}_m.$$

Of course, they form corresponding matrices:

$$L = \begin{bmatrix} L_{1,1} & L_{1,2} & \ldots & L_{1,n} \\ L_{2,1} & L_{2,2} & \ldots & L_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ L_{m,1} & L_{m,2} & \ldots & L_{m,n} \end{bmatrix}, \text{ same goes for } R \text{ and } O$$

An example of a single “if” statement:

$$\begin{aligned} \boldsymbol{\ell}_1 &= (0,0,1,0,0,0,0) \\ \boldsymbol{\ell}_2 &= (0,0,0,1,0,0,0) \\ \boldsymbol{\ell}_3 &= (0,0,1,0,0,0,0) \\ \boldsymbol{\ell}_4 &= (1,0,-1,0,0,0,0) \end{aligned} \qquad \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & \boxed{-1} & 0 & 0 & 0 & 0 \end{bmatrix}$$

Recap: QAP

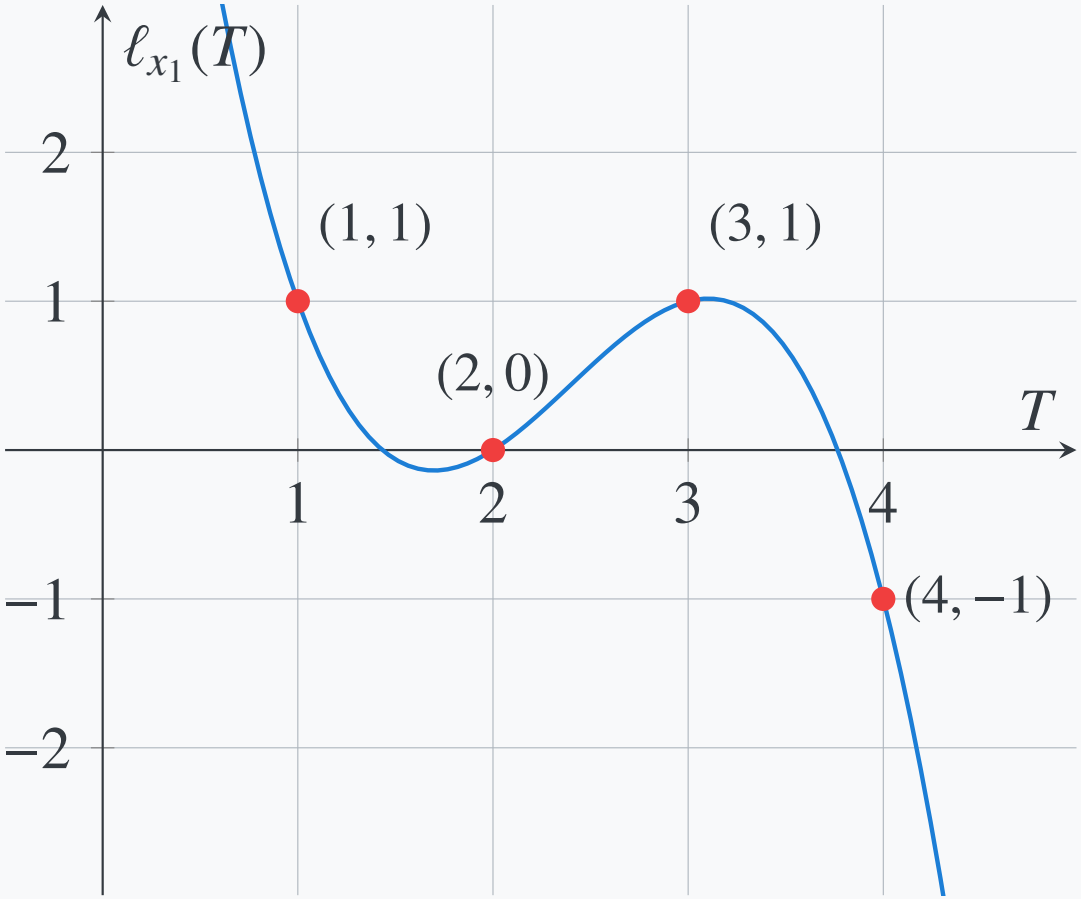


Illustration: The Lagrange interpolation polynomial for points $\{(1,1),(2,0),(3,1),(4,-1)\}$ visualized over $\mathbb{R}$.

Recap: QAP

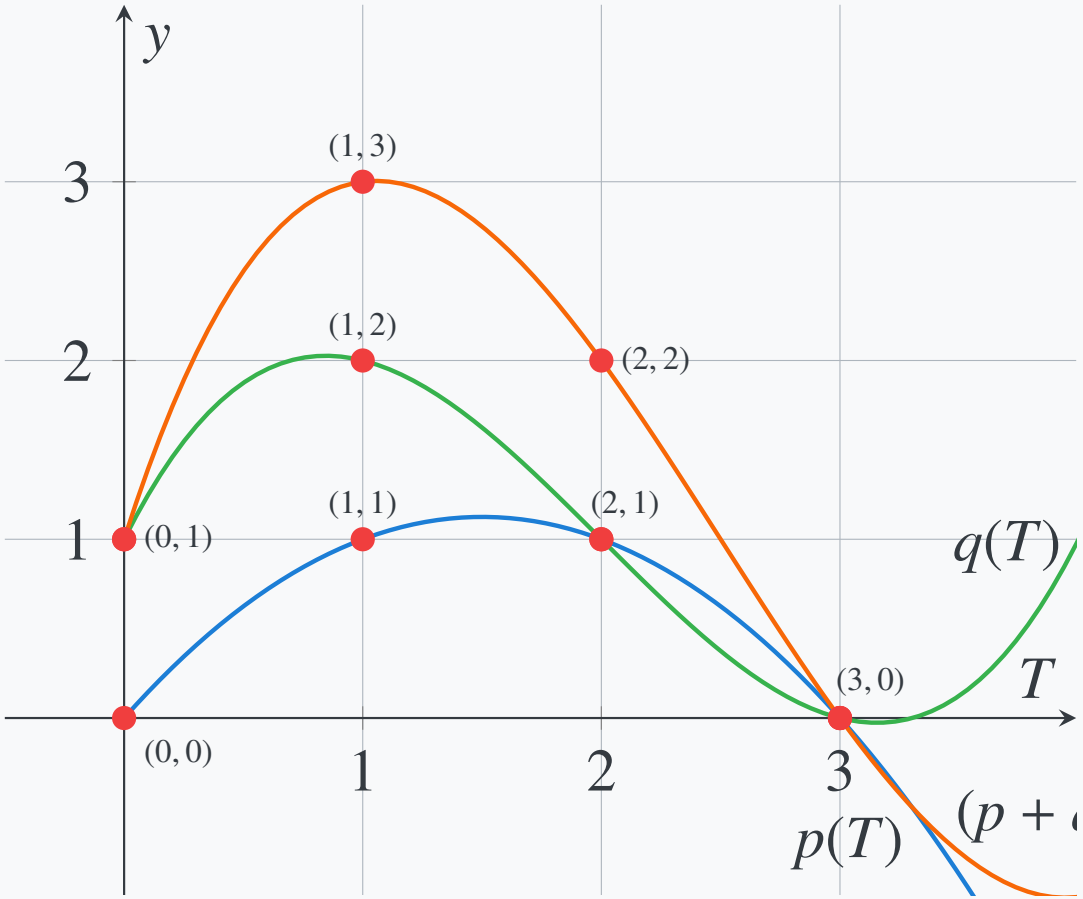


Figure: Addition of two polynomials

Recap: QAP

Now, using coefficients encoded with polynomials, we can build a constraint number $j \in \{1, \ldots, m\}$ in the next way:

$$\begin{aligned}(z_1\ell_1(j) + z_2\ell_2(j) + \cdots + z_n\ell_n(j))\times \\ \times(z_1 r_1(j) + z_2 r_2(j) + \cdots + z_n r_n(j)) = \\ =(z_1 o_1(j) + z_2 o_2(j) + \cdots + z_n o_n(j))\end{aligned}$$

Or written more concisely:

$$\left(\sum_{i=1}^{n} z_i \ell_i(T)\right) \times \left(\sum_{i=1}^{n} z_i r_i(T)\right) = \left(\sum_{i=1}^{n} z_i o_i(T)\right)$$

$$\ell(T) \times r(T) = o(T) \quad // \text{ on } \{1, \ldots, m\}$$

Recap: QAP

Now, we can define a **master polynomial** $m(T)$, that has zeros at all elements from the set $\Omega = \{1, \ldots, m\}$

$$m(T) = \ell(T) \times r(T) - o(T)$$

It means, that $m(T)$ can be divided by **vanishing polynomial** $z_\Omega(T)$ without a remainder!

$$z_\Omega(T) = \prod_{i=1}^{m}(T - i), \qquad h(T) = \frac{m(T)}{z_\Omega(T)} \text{ is a polynomial}$$

Encrypted Verification

Current Point

We've managed to encode into **a single polynomial** an entire computation (a program), of any size, independent of how much data it consumes.

Now, we need to figure out the protocol, how a prover can succinctly prove the knowledge of a correct witness for some circuit to a verifier, additionally, make it zero-knowledge and non-interactive.

Where the knowledge of the correct witness is a knowledge of the quotient polynomial $h(T)$.

$$m(T) = h(T) \times z_\Omega(T)$$

Remark

Further, for brevity, we will denote $z_\Omega(T)$ as $z(T)$.

Notation Preliminaries: Groups

In this section, we will use:

- ✓ Group of points on elliptic curve denoted as $\mathbb{G}$ of prime order q with a generator g.
- ✓ The symmetric pairing function $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$, where $(\mathbb{G}_T, \times)$ is a target group (typically, just a scalar from extension $\mathbb{F}_{p^k}$).

Recall

The core property of the pairing function e is the **bilinearity**:

$$e(g^\alpha, g^\beta) = e(g^{\alpha\beta}, g) = e(g, g^{\alpha\beta}) = e(g, g)^{\alpha\beta}.$$

Here, g^α is the same as "scalar multiplication of a generator by a scalar $\alpha \in \mathbb{Z}_q$".

Notation Preliminaries: QAP

Recall that the core equation to be proven is:

$$\underbrace{\left(\sum_{i=1}^{n} z_i \ell_i(T)\right)}_{\text{left wires encoding}} \times \underbrace{\left(\sum_{i=1}^{n} z_i r_i(T)\right)}_{\text{right wires encoding}} - \underbrace{\left(\sum_{i=1}^{n} z_i o_i(T)\right)}_{\text{output encodings}} = z(T)h(T)$$

Shortly, $\ell(T) \times r(T) - o(T) = z(T)h(T)$.

Naive Proof

Suppose, we are given a circuit C with a maximum degree d of polynomials used underneath.

Thus, all parties additionally know the target polynomial $z(T)$ and QAP polynomials $\{\ell_i(T)\}_{i\in[n]}, \{r_i(T)\}_{i\in[n]}, \{o_i(T)\}_{i\in[n]}$, where n is number of witness elements.

Prover

✓ Provides witness $\mathbf{z}$ to a Verifier.

Verifier

✓ Checks $\left(\sum_{i=1}^{n} z_i \ell_i(T)\right) \times \left(\sum_{i=1}^{n} z_i r_i(T)\right) = \left(\sum_{i=1}^{n} z_i o_i(T)\right)$ on Ω.

Naive Proof

- ✗ Succinct
- ✓ Non-Interactive
- ✗ Zero-Knowledge

The verifier could actually just run a program that represents a circuit C on witness data $\mathbf{z}$.



We, definitely, need to encrypt the witness data $\mathbf{z}$ somehow...

Encryption and KZG Commitment

Let us define the *encryption* operation as follows:

$$\text{Enc} : \mathbb{Z}_q \to \mathbb{G}, \quad \text{Enc}(x) := g^x$$

Essentially, $\text{Enc}(p(\tau))$ is the **KZG Commitment** $\text{com}(p)$.

Example

Consider the polynomial: $p(T) = T^2 - 5T + 2$, the encryption of $p(\tau)$ can be found as follows:

$$\text{Enc}(p(\tau)) = g^{p(\tau)} = g^{(\tau^2 - 5\tau + 2)} = \left(g^{\tau^2}\right)^1 \cdot \left(g^{\tau^1}\right)^{-5} \cdot \left(g^{\tau^0}\right)^2$$

Question

KZG Commitment requires encrypted powers of τ: $\{g^{\tau^i}\}_{i \in [d]}$. But where can the prover take them?

Trusted Setup

Trusted Party Setup

- ✓ Picks a random value $\tau \leftarrow\!\!\$\ \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ **Deletes** τ (toxic waste).
- ✓ **Outputs** prover parameters pp $\leftarrow \{g^{\tau^i}\}_{i\in[d]}$ and verifier parameters vp $\leftarrow \mathsf{com}(z)$.

This way, we can find the KZG commitment for each polynomial. For example:

$$\mathsf{com}(\ell) \triangleq g^{\ell(\tau)} = g^{\sum_{i=0}^{d} \ell_i \tau^i} = \prod_{i=0}^{d} (g^{\tau^i})^{\ell_i},$$

Encrypted Verification

Now, we can calculate the following KZG commitments (or, synonymously, encryptions):

$$g^{\ell(\tau)}, g^{r(\tau)}, g^{o(\tau)}, g^{h(\tau)}, g^{z(\tau)}$$

But how can we verify $h(T)z(T) = \ell(T)r(T) - o(T)$ in the encrypted space?

Well, first notice that, according to the **Schwartz-Zippel Lemma**, *with overwhelming probability* the check is equivalent to:

$$\ell(\tau)r(\tau) = z(\tau)h(\tau) + o(\tau).$$

So, we can check this equality as follows:

$$e(\mathrm{com}(\ell), \mathrm{com}(r)) = e(\mathrm{com}(z), \mathrm{com}(h)) \cdot e(\mathrm{com}(o), g),$$

Attempt #1: Encrypted Verification

Trusted Setup: $\tau \leftarrow\$ \mathbb{F}$, $\{g^{\tau^i}\}_{i\in[d]}$, $g^{z(\tau)}$, **delete**(τ).

✓ $h(T) = \frac{\ell(T)\times r(T) - o(T)}{z(T)}$.

✓ KZG commitments:

$\pi_L \leftarrow \mathsf{com}(\ell)$, $\pi_R \leftarrow \mathsf{com}(r)$,

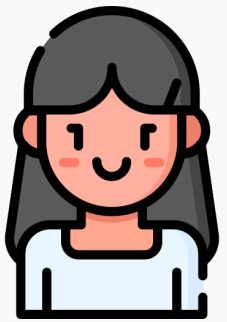
$\pi_O \leftarrow \mathsf{com}(o)$, $\pi_H \leftarrow \mathsf{com}(h)$,

✓ $e(\pi_L, \pi_R) \stackrel{?}{=} e(\mathsf{com}(z), \pi_H) \cdot e(\pi_O, g)$.



Prover $\mathcal{P}$

$\pi = (\pi_L, \pi_R, \pi_O, \pi_H)$



Verifier $\mathcal{V}$

Attempt #1: Encrypted Verification

- ✓ Succinct
- ✓ Non-Interactive
- ✓ Zero-Knowledge

- ✗ Does it work?

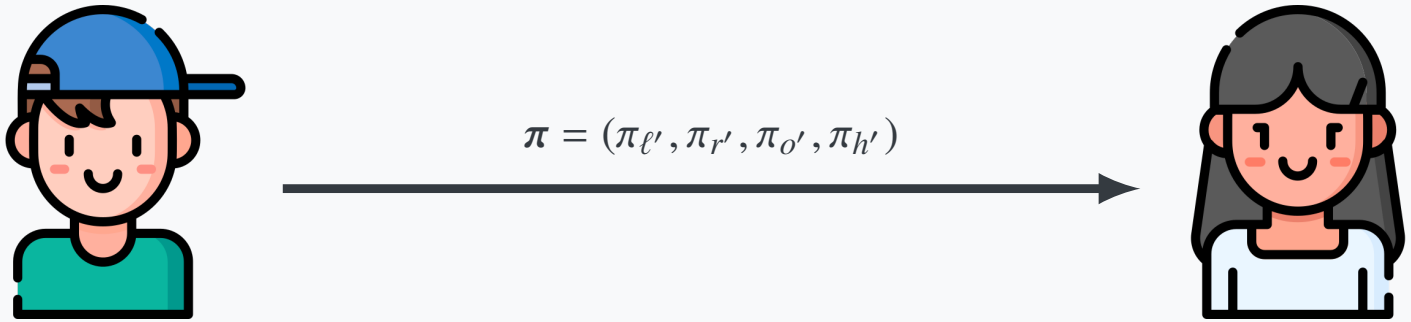
Why it doesn't work??

Trusted Setup: $\tau \leftarrow\!\$\ \mathbb{F}$, $\{g^{\tau^i}\}_{i\in[d]}$, $g^{z(\tau)}$, **delete**(τ).

- ✓ $h'(T) \leftarrow\!\$\ \mathbb{F}[T]$, $m'(T) = z(T) \times h'(T)$.
- ✓ Finds $\ell'(T), r'(T), o'(T)$ such that: $\ell'(T) \times r'(T) - o'(T) = m'(T)$
- ✓ KZG commitments:

$$\pi_{\ell'} \leftarrow \mathsf{com}(\ell'), \qquad \pi_{r'} \leftarrow \mathsf{com}(r'),$$
$$\pi_{o'} \leftarrow \mathsf{com}(o'), \qquad \pi_{h'} \leftarrow \mathsf{com}(h'),$$

- ✓ $e(\pi_{\ell'}, \pi_{r'}) \stackrel{?}{=} e(\mathsf{com}(z), \pi_{h'}) \cdot e(\pi_{o'}, g)$.



Problem

Prover isn't forced to use the values from the trusted setup.

Attempt #2: Proof Of Exponent

Trusted Setup: $\tau, \alpha \leftarrow\$ \mathbb{F}$, $\{\{g^{\tau^i}, g^{\alpha\tau^i}\}_{i\in[d]}\}$, $\{g^{z(\tau)}, g^{\alpha}\}$, **delete**$(\tau, \alpha)$.

✓ $h(T) = \frac{\ell(T)\times r(T) - o(T)}{z(T)}$.

✓ KZG commitments:

$$\pi_L \leftarrow g^{\ell(\tau)}, \quad \pi'_L \leftarrow g^{\alpha\ell(\tau)},$$
$$\pi_R \leftarrow g^{r(\tau)}, \quad \pi'_R \leftarrow g^{\alpha r(\tau)},$$
$$\pi_O \leftarrow g^{o(\tau)}, \quad \pi'_O \leftarrow g^{\alpha o(\tau)},$$
$$\pi_H \leftarrow g^{h(\tau)}, \quad \pi'_H \leftarrow g^{\alpha h(\tau)}.$$

✓ $e(\pi_L, \pi_R) \stackrel{?}{=} e(\mathsf{com}(z), \pi_H) \cdot e(\pi_O, g)$.

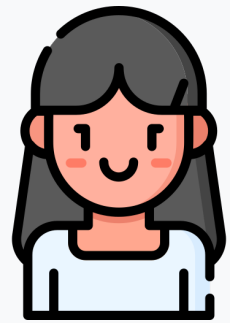
✓ Proof of Exponent:

$$e(\pi_L, g^\alpha) = e(\pi'_L, g),$$
$$e(\pi_R, g^\alpha) = e(\pi'_R, g),$$
$$e(\pi_O, g^\alpha) = e(\pi'_O, g),$$
$$e(\pi_H, g^\alpha) = e(\pi'_H, g).$$



Prover $\mathcal{P}$

$$\boldsymbol{\pi} = (\pi_L, \pi_R, \pi_O, \pi_H, \pi'_L, \pi'_R, \pi'_O, \pi'_H)$$



Verifier $\mathcal{V}$

Including PoE

- ✓ Succinct
- ✓ Non-Interactive
- ✓ Zero-Knowledge
- ✗ Sound

Problem

There is no guarantee that the same witness $\mathbf{z}$ was used to calculate all the commitments $\pi_L, \pi_R, \pi_O, \pi_H$.

Make It Sound

Additional Optimization

Recall that:

$$\ell(T) = \sum_{i=1}^{n} z_i \ell_i(T), \quad r(T) = \sum_{i=1}^{n} z_i r_i(T), \quad o(T) = \sum_{i=1}^{n} z_i o_i(T).$$

Here public data is:

$$\{\ell_i(T)\}_{i\in[n]}, \{r_i(T)\}_{i\in[n]}, \{o_i(T)\}_{i\in[n]}$$

Moreover, it's defined only by the circuit and trusted setup, thus, it can be calculated before proof generation as a part of the trusted setup.

Additional Optimization

Updated Trusted Setup:

$$\{g^{\tau^i}\}_{i\in[d]}, \quad \{g^{\alpha\tau^i}\}_{i\in[d]},$$
$$\{g^{\ell_i(\tau)}\}_{i\in[n]}, \quad \{g^{\alpha\ell_i(\tau)}\}_{i\in[n]},$$
$$\{g^{r_i(\tau)}\}_{i\in[n]}, \quad \{g^{\alpha r_i(\tau)}\}_{i\in[n]},$$
$$\{g^{o_i(\tau)}\}_{i\in[n]}, \quad \{g^{\alpha o_i(\tau)}\}_{i\in[n]}$$

Consider the polynomial $\ell(T) = \sum_{i=1}^{n} z_i \ell_i(T)$.

$\mathcal{P}$ can compute the KZG commitment π_L and its PoE π'_L as follows:

$$\pi_L \triangleq g^{\ell(\tau)} = g^{\sum_{i=1}^{n} z_i \ell_i(\tau)} = \prod_{i=1}^{n} (g^{\ell_i(\tau)})^{z_i},$$

$$\pi'_L \triangleq g^{\alpha\ell(\tau)} = g^{\alpha \sum_{i=1}^{n} z_i \ell_i(\tau)} = \prod_{i=1}^{n} (g^{\alpha\ell_i(\tau)})^{z_i}.$$

Witness Consistency Check

To prove that the same $\mathbf{z}$ is used in all commitments, we need some "checksum" term that will somehow combine all polynomials $\ell(T)$, $r(T)$, and $o(T)$ with the witness $\mathbf{z}$.

We can do so by proving the next simple statement:

$$g^{\ell(\tau)+r(\tau)+o(\tau)} = \prod_{i=1}^{n} \left(g^{\ell_i(\tau)+r_i(\tau)+o_i(\tau)}\right)^{z_i}$$

And we already know how to do that — PoE!

Witness Consistency Check

Let us introduce one more coefficient...

$$\beta \leftarrow\$ \mathbb{F}$$

Extended trusted setup contains additional values:

$$g^{\beta}, \quad \{g^{\beta(\ell_i(\tau)+r_i(\tau)+o_i(\tau))}\}_{i\in[n]}$$

Prover needs to calculate π_β:

$$\pi_\beta \leftarrow g^{\beta(\ell(\tau)+r(\tau)+o(\tau))} = \prod_{i=1}^{n} \left(g^{\beta(\ell_i(\tau)+r_i(\tau)+o_i(\tau))}\right)^{z_i}$$

And easy check for verifier:

$$e(\pi_L \pi_R \pi_O, g^{\beta}) = e(\pi_\beta, g).$$

One more time... that doesn't work

If the witness is consistent, the following condition must hold:

$$(z_{\ell,i}\ell_i(\tau) + z_{r,i}r_i(\tau) + z_{o,i}o_i(\tau))\beta = z_{\beta,i}\beta(\ell_i(\tau) + r_i(\tau) + o_i(\tau)) \quad \textit{// for all } i \in [n]$$

But, what if $\ell_i \equiv r_i$. Let us call them q, thus:

$$(z_{\ell,i} + z_{r,i})q + z_{o,i}o_i(\tau) = z_{\beta,i}(2q + o_i(\tau)) \quad \textit{// for all } i \in [n]$$

The adversary can choose $z_{\ell,i}$, $z_{r,i}$ and $z_{o,i}$ such that:

$$z_{\beta,i} := z_{o,i} \quad \text{and} \quad z_{\ell,i} = 2z_{o,i} - z_{r,i}$$

Example

$$z_\beta = z_o = 5, \quad z_\ell = 7, \quad z_r = 3$$

$$(7 + 3)q + 5o(\tau) = 5(2q + o(\tau))$$

$$10q + 5o(\tau) = 10q + 5o(\tau)$$

More coefficients!

The main problem is that if $r_i \equiv \ell_i$ then $\beta r_i \equiv \beta \ell_i$.

Let us fix that by introducing separate β coefficients for ℓ, r and o.

$$(\beta_L, \beta_R, \beta_O) \leftarrow\$ \mathbb{F}^3$$

Therefore, $\beta_R r_i \neq \beta_L \ell_i$ even if $r_i \equiv \ell_i$, so the previous hack doesn't work.

So, finally, the trusted setup is updated with:

$$g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, \{g^{\beta_L \ell_i(\tau) + \beta_R r_i(\tau) + \beta_O o_i(\tau)}\}_{i \in [n]}$$

Verification:

$$e(\pi_L, g^{\beta_L}) \cdot e(\pi_R, g^{\beta_R}) \cdot e(\pi_O, g^{\beta_O}) = e(\pi_\beta, g)$$

Even more coefficients!

As the adversary has access to the public g^{β_L}, g^{β_R}, g^{β_O} he still can cheat the verifier by calculating a modified π_β.

Example

Consider a constraint $z_1 \times z_1 = z_2$. Let us try to assign 2 and 5 for z_1 in a single constraint. As $2 \times 5 = 10$, z_2 should contain the value 10.

$$\mathbf{z} = (z_1, z_2) = (2, 10)$$

The next QAP can be built:

$$\ell(T) = 2\ell_1(T) + 10\ell_2(T)$$
$$r(T) = 2r_1(T) + 3 + 10r_2(T)$$
$$o(T) = 2o_1(T) + 10o_2(T)$$

Compute π_β as:

$$(g^{\beta_L \ell_1(\tau) + \beta_R r_1(\tau) + \beta_O o_1(\tau)})^2 \cdot (g^{\beta_R})^3 \cdot (g^{\beta_L \ell_2(\tau) + \beta_R r_2(\tau) + \beta_O o_2(\tau)})^{10}$$

Even more coefficients!

To prevent this, let us introduce... one more coefficient!

$$\gamma \leftarrow\$ \mathbb{F}$$

So, finally... the trusted setup is updated with:

$$g^{\gamma}, g^{\beta_L \gamma}, g^{\beta_R \gamma}, g^{\beta_O \gamma}, \{g^{\beta_L \ell_i(\tau) + \beta_R r_i(\tau) + \beta_O o_i(\tau)}\}_{i \in [n]}$$

Proving process isn't changed, unlike verification:

$$e(\pi_L, g^{\beta_L \gamma}) \cdot e(\pi_R, g^{\beta_R \gamma}) \cdot e(\pi_O, g^{\beta_O \gamma}) = e(\pi_\beta, g^{\gamma})$$

That makes it infeasible to cheat.

Attempt #3: Sound SNARK Protocol

Trusted Setup: $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \leftarrow\$ \mathbb{F}$, $\{\{g^{\tau^i}, g^{\alpha\tau^i}\}_{i\in[d]}$, $\{g^{\beta_L \ell_i(\tau)+\beta_R r_i(\tau)+\beta_O o_i(\tau)}\}_{i\in[n]}\}$, $\{g^{z(\tau)}, g^{\alpha}, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L\gamma}, g^{\beta_R\gamma}, g^{\beta_O\gamma}, g^{\gamma}\}$, **delete**$(\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma)$.

- ✓ $h(T) = \frac{\ell(T)\times r(T) - o(T)}{z(T)}$.
- ✓ KZG commitments:

$$\pi_L \leftarrow g^{\ell(\tau)}, \quad \pi'_L \leftarrow g^{\alpha\ell(\tau)},$$
$$\pi_R \leftarrow g^{r(\tau)}, \quad \pi'_R \leftarrow g^{\alpha r(\tau)},$$
$$\pi_O \leftarrow g^{o(\tau)}, \quad \pi'_O \leftarrow g^{\alpha o(\tau)},$$
$$\pi_H \leftarrow g^{h(\tau)}, \quad \pi'_H \leftarrow g^{\alpha h(\tau)}.$$

- ✓ $\pi_\beta \leftarrow g^{\beta_L \ell(\tau)+\beta_R r(\tau)+\beta_O o(\tau)}$

- ✓ $e(\pi_L, \pi_R) \stackrel{?}{=} e(\mathsf{com}(z), \pi_H) \cdot e(\pi_O, g)$.
- ✓ Proof of Exponent:

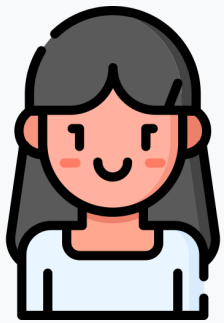
$$e(\pi_L, g^\alpha) = e(\pi'_L, g),$$
$$e(\pi_R, g^\alpha) = e(\pi'_R, g),$$
$$e(\pi_O, g^\alpha) = e(\pi'_O, g),$$
$$e(\pi_H, g^\alpha) = e(\pi'_H, g).$$

- ✓ $e(\pi_L, g^{\gamma\beta_L}) \cdot e(\pi_R, g^{\gamma\beta_R}) \cdot e(\pi_O, g^{\gamma\beta_O}) = e(\pi_\beta, g^\gamma)$



Prover $\mathcal{P}$

$$\pi = (\pi_L, \pi_R, \pi_O, \pi_H, \pi'_L, \pi'_R, \pi'_O, \pi'_H, \pi_\beta)$$



Verifier $\mathcal{V}$

Make it Zero-Knowledge

Why do we need modifications?

Question

- According to the security assumptions, given any element of our proof π, it is impossible to restore the witness $\mathbf{z}$.
- Well, that is true, but does it guarantee that no other information about $\mathbf{z}$ is **leaked**?

It does not! For example, given π, I can check if $\ell(T) = 10r(T)$.
How? Simply check

$$\pi_L \stackrel{?}{=} \pi_R^{10}.$$

This works since $\pi_L = g^{\ell(\tau)}$ and $\pi_R = g^{r(\tau)}$, so

$$\pi_L = \pi_R^{10} \Leftrightarrow g^{\ell(\tau)} = g^{10r(\tau)} \Leftrightarrow \ell(\tau) = 10r(\tau) \Leftrightarrow \ell(T) = 10r(T)$$

How do we fix that? Analysis

Analysis

- Currently, proof consists of four ingredients: $\pi_L, \pi_R, \pi_O, \pi_H$, and their corresponding PoEs.
- But, current construction creates some "connection" between these ingredients. For example, $\pi_L^{101} = \pi_R$ reveals whether $101\ell(T) = r(T)$ or $\pi_L \pi_O = \pi_R$ whether $\ell(T) + o(T) = r(T)$.
- Therefore, the prover must add some "noise" to the proof, so that the verifier can't use the proof to extract any information about the witness. The randomness of prover is kept secret.
- Simultaneously, this noise would still make the same proof checkable by already defined verification equations.

Question

So how do we represent such "noise" while preserving "homomorphic" properties of the proof?

How do we fix that? Doing stuff

Idea #1

Let prover pick random values $\delta_L, \delta_R, \delta_O, \delta_H \leftarrow\$ \mathbb{F}$ and calculate the "distorted" values:

$$\pi_L \leftarrow g^{\ell(\tau)+\delta_L}, \quad \pi_R \leftarrow g^{r(\tau)+\delta_R}, \quad \text{same for } \pi_O, \pi_H$$

Problem (informally): To make previous verification mechanism work, δ_H must be picked "carefully". However, it's impossible to do so with this construction.

Idea #2

Let prover pick random values $\delta_L, \delta_R, \delta_O \leftarrow\$ \mathbb{F}$ and calculate:

$$\pi_L \leftarrow g^{\ell(\tau)+\delta_L z(\tau)}, \quad \pi_R \leftarrow g^{r(\tau)+\delta_R z(\tau)}, \quad \pi_O \leftarrow g^{o(\tau)+\delta_O z(\tau)}$$

Making Idea Practical

This expression can be easily evaluated. For example,

$$\pi_L = g^{\ell(\tau)} \cdot \left(g^{z(\tau)}\right)^{\delta_L} = \mathrm{com}(\ell) \cdot \mathrm{com}(z)^{\delta_L}$$

Problem

However, the verifier can't check the equation $e(\pi_L, \pi_R) = e(\mathrm{com}(z), \pi_H) \cdot e(\pi_O, g)$ anymore.

Idea #3

Our "noise" must be controlled! This can be achieved by calculating π_H as $g^{h(\tau)+\Delta_H}$ where Δ_H depends on $\delta_L, \delta_R, \delta_O$ (and possibly other public parameters).

Making Idea Practical

Check $e(\pi_L, \pi_R) = e(\text{com}(z), \pi_H) \cdot e(\pi_O, g)$ is now equivalent to:

$$(\ell(T) + \delta_L z(T))(r(T) + \delta_R z(T)) = (h(T) + \Delta_H)z(T) + (o(T) + \delta_O z(T)),$$

which, by expanding, gives us the following equation:

$$\begin{aligned} \cancel{\ell(T)r(T)} + \delta_R \ell(T)z(T) + \delta_L z(T)r(T) + \delta_L \delta_R z(T)^2 \\ = \cancel{h(T)z(T) + o(T)} + \Delta_H z(T) + \delta_O z(T) \end{aligned}$$

where we can cancel out $\ell(T)r(T)$ and $h(T)z(T) + o(T)$ terms since they are equal based on initial construction. This way, we get the following expression for Δ_H:

$$\boxed{\Delta_H = \delta_O + \delta_R \ell(T) + \delta_L r(T) + \delta_L \delta_R z(T)}$$

Witness Consistency Proof

Finally, let us not forget about π_β! Previously, we had:

$$\pi_\beta = g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}$$

Now, we change:

- $\ell(\tau) \mapsto \ell(\tau) + \delta_L z(\tau)$
- $r(\tau) \mapsto r(\tau) + \delta_R z(\tau)$
- $o(\tau) \mapsto o(\tau) + \delta_O z(\tau)$

Therefore, our new π_β becomes:

$$\pi_\beta = \left(g^{\beta_L z(\tau)}\right)^{\delta_L} \left(g^{\beta_R z(\tau)}\right)^{\delta_R} \left(g^{\beta_O z(\tau)}\right)^{\delta_O} g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}$$

Overall Protocol

Trusted Setup: $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \leftarrow\$ \mathbb{F}$, $\{\{g^{\tau^i}, g^{\alpha\tau^i}\}_{i\in[d]}$, $\{g^{\beta_L \ell_i(\tau)}, g^{\beta_R r_i(\tau)}, g^{\beta_O o_i(\tau)}\}_{i\in[n]}\}$, $\{g^{z(\tau)}, g^{\alpha}, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L\gamma}, g^{\beta_R\gamma}, g^{\beta_O\gamma}, g^{\gamma}\}$, **delete**$(\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma)$.

- ✓ $h(T) = \frac{\ell(T)\times r(T) - o(T)}{z(T)}$.
- ✓ Sample $\delta_L, \delta_R, \delta_O \leftarrow\$ \mathbb{F}$, compute:

$$\pi_L \leftarrow g^{\ell(\tau)}(g^{z(\tau)})^{\delta_L}, \pi'_L \leftarrow g^{\alpha\ell(\tau)}(g^{\alpha z(\tau)})^{\delta_L},$$
$$\pi_R \leftarrow g^{r(\tau)}(g^{z(\tau)})^{\delta_R}, \pi'_R \leftarrow g^{\alpha r(\tau)}(g^{\alpha z(\tau)})^{\delta_R},$$
$$\pi_O \leftarrow g^{o(\tau)}(g^{z(\tau)})^{\delta_O}, \pi'_O \leftarrow g^{\alpha o(\tau)}(g^{\alpha z(\tau)})^{\delta_O},$$
$$\pi_H \leftarrow g^{h(\tau)}(g^{\delta_O})(g^{r(\tau)})^{\delta_L}(g^{\ell(\tau)})^{\delta_R}(g^{z(\tau)})^{\delta_L\delta_R}$$

$\pi_\beta = \ldots$

- ✓ $e(\pi_L, \pi_R) \stackrel{?}{=} e(\mathsf{com}(z), \pi_H) \cdot e(\pi_O, g)$.
- ✓ Proof of Exponent:

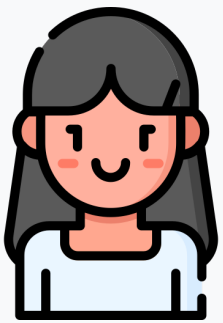
$$e(\pi_L, g^\alpha) = e(\pi'_L, g),$$
$$e(\pi_R, g^\alpha) = e(\pi'_R, g),$$
$$e(\pi_O, g^\alpha) = e(\pi'_O, g),$$
$$e(\pi_H, g^\alpha) = e(\pi'_H, g).$$

- ✓ $e(\pi_L, g^{\gamma\beta_L}) \cdot e(\pi_R, g^{\gamma\beta_R}) \cdot e(\pi_O, g^{\gamma\beta_O}) = e(\pi_\beta, g^\gamma)$



Prover $\mathcal{P}$

$$\boldsymbol{\pi} = (\pi_L, \pi_R, \pi_O, \pi_H, \pi'_L, \pi'_R, \pi'_O, \pi'_H, \pi_\beta)$$



Verifier $\mathcal{V}$

Real Protocols

Public and Private Inputs

So far, the verifier did not use the public statement at all. Split the indices of the extended witness $\mathbf{z}$ into $\mathcal{I}_{\text{io}}$ (input/output, public) and $\mathcal{I}_{\text{mid}}$ (intermediate, private), with $\mathcal{I}_{\text{mid}} \cap \mathcal{I}_{\text{io}} = \emptyset$, $\mathcal{I}_{\text{mid}} \cup \mathcal{I}_{\text{io}} = \{1, \ldots, n\}$.

$$\pi_L = \underbrace{g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i \ell_i(\tau)}}_{\text{new } \pi_L} \times \underbrace{g^{\sum_{i \in \mathcal{I}_{\text{io}}} z_i \ell_i(\tau)}}_{\pi_{\ell,\text{io}}}.$$

Remark

The prover computes π_L, π_R, π_O only over $\mathcal{I}_{\text{mid}}$, while the verifier restores $\pi_L^* \leftarrow \pi_L \prod_{i \in \mathcal{I}_{\text{io}}} (g^{\ell_i(\tau)})^{z_i}$ (and similarly π_R^*, π_O^*) from the public values and uses them in the polynomial equality check.

Complexity of the Basic Protocol

Proposition (Complexity of the Basic Protocol)

Suppose circuit consists of n gates. Then, the complexity of the basic protocol is as follows:

- **Proof Size:** $O(1)$ — constant number of group elements.
- **Setup Time:** $O(n)$ — calculating powers of τ, evaluations at τ.
- **Prover Time:** $O(n \log n)$ — using FFT and wise choice of Ω.
- **Verifier Time:** $O(1)$ — constant number of pairings.

However, $O(1)$ is not very descriptive for proof and verifier complexities, so let us provide a more detailed analysis.

- **Proof Size:** 9 $\mathbb{G}$ group elements.
- **Verifier Time:** 15 pairings.

We can do better!

Pinocchio Protocol

Idea

In toxic waste, include $\rho_L, \rho_R \leftarrow\$ \mathbb{F}$, set $\rho_O \leftarrow \rho_L \rho_R$, and define the following generators:

$$g_L \leftarrow g^{\rho_L}, \quad g_R \leftarrow g^{\rho_R}, \quad g_O \leftarrow g^{\rho_O}$$

Reason

Such choice of generators reduces 15 pairings to **11 pairings**. Additionally, we have only **8 group elements** in the proof.

Groth16 Protocol

Idea: Generic Group Model

Use **Generic Group Model** (GGM) technique. Simply put, GGM allows the adversary to only make **oracle requests** to compute the group operations. For example, having a set $\{g^{\alpha r_i(\tau)}\}_{i\in[d]}$, adversary can compute only linear combinations of these values. In the particular case of Groth16, instead of considering $\ell_i(T)$, $r_i(T)$, and $o_i(T)$ separately, we construct their linear combinations as $Q_i(T) := \beta\ell_i(T) + \alpha r_i(T) + o_i(T)$, where α and β are toxic parameters.

💬 *Asymmetric Pairing*

Groth16 uses the asymmetric pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ with generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$: it is more efficient than the symmetric one.

Groth16 Protocol: Setup Procedure

Sample $\alpha, \beta, \gamma, \delta, \tau \leftarrow\$ \mathbb{F}$. The proving key is formed as follows:

$$\mathsf{pp} \leftarrow \Big(g_1^{\alpha}, g_1^{\beta}, g_1^{\delta}, \Big\{g_1^{\tau^i}, g_1^{(\beta\ell_i(\tau)+\alpha r_i(\tau)+o_i(\tau))/\gamma}, g_1^{\tau^i z(\tau)/\delta}\Big\}_{i\in[n]},$$
$$g_2^{\beta}, g_2^{\delta}, g_2^{\gamma}, \{g_2^{\tau^i}\}_{i\in[d]}\Big)$$

Groth16 Protocol: Proving Procedure

Sample random $\delta_L, \delta_R \leftarrow\$ \mathbb{F}$ and compute the following values:

$$\pi_L \leftarrow g_1^{\alpha+\sum_{i=1}^{n} z_i \ell_i(\tau)+\delta_L \delta}, \quad \pi_R \leftarrow g_2^{\beta+\sum_{i=1}^{n} z_i r_i(\tau)+\delta_R \delta},$$

$$\pi_O \leftarrow g_1^{\frac{Q_{\mathsf{mid}}(\tau)+h(\tau)z(\tau)}{\delta}+\ell\delta_R+r\delta_L-\delta_L\delta_R\delta},$$

where by Q_{mid} we denoted the following expression:

$$Q_{\mathsf{mid}}(\tau) = \sum_{i \in \mathcal{I}_{\mathsf{mid}}} z_i Q_i(\tau) = \sum_{i \in \mathcal{I}_{\mathsf{mid}}} z_i(\beta \ell_i(\tau) + \alpha r_i(\tau) + o_i(\tau))$$

Groth16 Protocol: Verification Procedure

The verifier first calculates the following value:

$$\pi_{\mathsf{io}} \leftarrow g_1^{\sum_{i \in \mathcal{I}_{\mathsf{io}}} z_i(\beta \ell_i(\tau) + \alpha r_i(\tau) + o_i(\tau))/\gamma},$$

and then checks the following single condition:

$$e(\pi_L, \pi_R) = e(g_1^{\alpha}, g_2^{\beta}) e(\pi_{\mathsf{io}}, g_2^{\gamma}) e(\pi_O, g_2^{\delta})$$

Note

$e(g_1^{\alpha}, g_2^{\beta})$ can be additionally hard-coded in the verifier, thus reducing the number of pairings to 3. Finally, the proof's size is now reduced to 3 group elements: two from $\mathbb{G}_1$, and one from $\mathbb{G}_2$.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream